Department of Computer Science and Engineering

Database Management Systems

Lab 06 | Nested and Correlated Queries in SQL

Smart Hostel Management System – A Case Study

Objective

To understand and implement nested and correlated queries in SQL, and to retrieve data using subqueries that depend on the results of an outer query, in MySQL, on the Smart Hostel Management System database you designed in Lab 2 (ER Model) and implemented in Lab 3 (Relational Schema & DDL).

Outcome

By completing this lab, students will learn to:

- Identify the situations in which a subquery is required instead of a simple SELECT statement.
- Write single-row (scalar) subqueries and use them with comparison operators.
- Write multi-row subqueries and use them with the IN and NOT IN operators.
- Use the ANY and ALL quantifiers with subqueries.
- Use EXISTS and NOT EXISTS to test whether related rows exist.
- Write correlated subqueries in which the inner query refers to a column of the outer query.
- Combine subqueries with the aggregate functions COUNT, SUM, AVG, MIN and MAX.
- Use a subquery in the FROM clause as a derived table.
- Distinguish a nested (non-correlated) subquery from a correlated subquery and explain when each is appropriate.
- Verify query results in MySQL and record the executed query together with its output.

Before You Begin

- Use the smart_hostel database created in Lab 3, which already holds the data used below. Select it with USE smart_hostel; before you begin. Lab 6 adds no tables and changes no data — do not redesign the case study or change the schema.
- Before executing any command, set the prompt in the MySQL command line client so that your SRN and the active database are visible on every line: prompt (\U) [\d] > — in the screenshots the prompt then appears as <SRN>@smart_hostel> on every input line.
- Execute every SQL query in MySQL and verify the result before recording the output. For each question, record the SQL query you wrote together with a screenshot of its execution and its output. Your SRN must be clearly visible inside every screenshot.
- Write each query with a comment giving its question number, for example -- Task 1.1, before the query itself.
- Write your queries in notepad after executing in MySQL; you can save the notepad file as .sql for submission.

Deliverables

- A single PDF containing, in order, each of the 22 questions with the SQL query you wrote and a screenshot of its output, plus the SQL file with the complete code which you implemented. Label each screenshot with its question number.
- File names:
    - <SRN>_NestedCorrelated_Lab06.pdf
    - <SRN>_NestedCorrelated_Lab06.sql
- Keep the database as it is after submission — the same schema and data will be reused in the later labs on procedures, functions, triggers and transactions.

Tasks to be executed:

- **Task 1:** Nested (non-correlated) queries — Questions 1.1 – 1.11
- **Task 2:** Correlated queries — Questions 2.1 – 2.11
- 22 questions in total; every query is written on the existing Lab 3 schema.

Schema Reference

The questions below refer only to the tables and columns listed here. Use these names exactly. This is the schema created by Lab 3.

Table	Columns	Primary key
HOSTEL	Hostel_ID, Hostel_Name, Location, Capacity	Hostel_ID
STAFF	Staff_ID, Name, Salary, Designation, Hostel_ID, Supervisor_ID	Staff_ID
ROOM	Hostel_ID, Room_Number, Floor	(Hostel_ID, Room_Number)
BED	Hostel_ID, Room_Number, Bed_Number, Availability	(Hostel_ID, Room_Number, Bed_Number)
STUDENT	Student_ID, First_Name, Middle_Name, Last_Name, Gender, Date_of_Birth, Email_Address	Student_ID
STUDENT_PHONE	Student_ID, Phone_Number	(Student_ID, Phone_Number)
FACILITY	Facility_ID, Facility_Name, Operating_Hours, Hostel_ID	Facility_ID
USES	Student_ID, Facility_ID	(Student_ID, Facility_ID)
ALLOCATE	Hostel_ID, Room_Number, Student_ID, Allocation_Date, Check_in_Date, Check_out_Date	(Hostel_ID, Room_Number, Student_ID)
PAYMENT	Payment_ID, Amount, Payment_Type, Payment_Date, Student_ID	Payment_ID
RECEIPT	Receipt_ID, Receipt_Date, Payment_ID	Receipt_ID

Identifiers follow a three-digit convention throughout: H001–H005, ST001–ST025, S001–S060, F001–F016, P001–P060, R001–R060. Age is not stored; it is derived from Date_of_Birth.

Task 1: Nested Queries

A nested (non-correlated) subquery is evaluated once, on its own, and supplies a value or a set of values to the outer query. It does not refer to any column of the outer query. Write each of the following queries using a nested subquery.

1.1 Display the Hostel_ID, Hostel_Name and Capacity of every hostel whose capacity is greater than the average capacity of all hostels. The average capacity must be obtained from the database using a subquery and must not be typed as a constant.

1.2 Display the Student_ID, first name and last name of every student who has made a payment whose amount is equal to the highest payment amount recorded in the PAYMENT table.

1.3 Display the Student_ID, first name and last name of every student who has made at least one payment greater than the average amount of all payments in the PAYMENT table. Each student must appear only once in the result.

1.4 Display the Staff_ID, name, designation and salary of every staff member whose salary is greater than the average salary of all staff members.

1.5 Display the Student_ID, first name and last name of every student who is allocated to the hostel that has the maximum capacity among all hostels. Use the SQL IN operator with a subquery.

1.6 Display the Student_ID, first name and last name of every student who has never made a payment. Use the SQL NOT IN operator with a subquery.

1.7 Display the Staff_ID, name, salary and Hostel_ID of every staff member whose salary is greater than the salary of every staff member working in hostel H003. Use the SQL ALL operator with a subquery.

1.8 Display the Staff_ID, name, salary and Hostel_ID of every staff member whose salary is greater than at least one salary among the staff members working in hostel H004. Use the SQL ANY operator with a subquery.

1.9 Display the Student_ID, first name and last name of every student who has made a payment equal to the minimum payment amount recorded in the PAYMENT table.

1.10 For every student who has made at least one payment, display the Student_ID, first name, last name and the total amount paid by that student. Restrict the output to students whose total payment is greater than the average of the per-student totals. Obtain that average using a nested query.

1.11 Display the Student_ID, first name and last name of every student who has made a payment greater than the average payment amount of the students allocated to hostel H002. Each student must appear only once in the result.

Task 2: Correlated Queries

In a correlated subquery the inner query refers to a column of the outer query, so the inner query is re-evaluated for every row that the outer query considers. Write each of the following queries using a correlated subquery.

2.1 Display the Staff_ID, name, salary and Hostel_ID of every staff member whose salary is greater than the average salary of the staff members working in the same hostel. The average must be recomputed for each staff member using a correlated subquery.

2.2 Display the Student_ID, first name and last name of every student who has made at least one payment that is greater than that same student's own average payment amount. Each student must appear only once in the result.

2.3 Display the Student_ID, first name and last name of every student who has made at least one payment of more than 30000. Use the SQL EXISTS operator with a correlated subquery.

2.4 Display the Student_ID, first name and last name of every student who has never made any payment. Use the SQL NOT EXISTS operator with a correlated subquery, and compare the result with the answer obtained in question 1.6.

2.5 Display the Student_ID, first name and last name of every student who uses at least one facility. Use the SQL EXISTS operator with a correlated subquery.

2.6 Display the Staff_ID, name, salary and Hostel_ID of every staff member who earns the highest salary in the hostel where they work. Use a correlated subquery.

2.7 Display the Hostel_ID, Room_Number and Student_ID of every allocation that belongs to a room in which more than one student is allocated. Use a correlated COUNT subquery.

2.8 Display the Student_ID, first name, last name and number of payments of every student whose number of payments is greater than the average number of payments made per paying student. Use a correlated subquery to count the payments of each student.

2.9 Display the Staff_ID, name and salary of every staff member whose salary is not shared by any other staff member. Use a correlated subquery that counts how many staff members have the same salary, and list the result in descending order of salary.

2.10 Display the Student_ID, first name and last name of every student who uses every facility that student S001 uses. Use the double NOT EXISTS pattern.

2.11 Display the Student_ID, first name, last name and number of receipts of every student for whom more than one receipt has been generated. Use a correlated subquery over the RECEIPT and PAYMENT tables.